

Refatoração do TaskList — Tutorial Completo

O problema

O componente `TaskList` em seu estado atual (~220 linhas) acumula três responsabilidades distintas:

1. **Buscar e manipular dados** — `fetch`, `setTasks`, `handleDelete`, `handleComplete`, `onEditSubmit`
2. **Controlar a UI do modal** — `useRef`, `useForm`, `editingTask`, `handleEdit`
3. **Renderizar o JSX** — lista de tarefas, modal de edição, botão concluir

O **princípio de responsabilidade única** diz que cada peça de código deve ter um único motivo para mudar. Com tudo junto, qualquer alteração de dado, de visual ou de modal toca o mesmo arquivo, aumentando o risco de introduzir bugs.

Estrutura de pastas após a refatoração

```
src/  
  components/  
    taskBook/  
      task-list.jsx      ← componente simplificado (~70 linhas)  
      edit-task-modal.jsx ← componente novo (modal de edição)  
  hooks/  
    use-tasks.js         ← hook customizado (lógica de dados)
```

Extração 1 — Hook `useTasks`

Arquivo: `src/hooks/use-tasks.js`

Concentra toda a lógica de estado e comunicação com a API. Qualquer componente que precise manipular tarefas pode importar esse hook sem duplicar código.

```
import { useEffect, useState } from "react"  
  
export const useTasks = () => {  
  const [tasks, setTasks] = useState([])  
  const [selectedTask, setSelectedTask] = useState([])  
  
  useEffect(() => {  
    const fetchTasks = async () => {  
      const response = await fetch("http://localhost:8000/tasks")  
      const data = await response.json()  
      setTasks(data)  
    }  
    fetchTasks()  
  }, [])  
}
```

```
const onEditSubmit = async (editingTask, data) => {
  const response = await fetch(`http://localhost:8000/tasks/${editingTask._id}`,
{
  method: "PATCH",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(data),
})

  if (response.ok) {
    const updatedTask = await response.json()
    setTasks((current) =>
      current.map((t) => (t._id === editingTask._id ? updatedTask : t))
    )
  } else {
    const error = await response.json()
    console.error(error)
  }
}

const handleDelete = async (taskId) => {
  const response = await fetch(`http://localhost:8000/tasks/${taskId}`, {
    method: "DELETE",
  })

  if (response.ok) {
    setTasks((current) => current.filter((t) => t._id !== taskId))
  } else {
    const error = await response.json()
    console.error(error)
  }
}

const handleComplete = async () => {
  await Promise.all(
    selectedTask.map((id) =>
      fetch(`http://localhost:8000/tasks/${id}`, {
        method: "PATCH",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ isCompleted: true }),
      })
    )
  )

  setTasks((current) => current.filter((t) => !selectedTask.includes(t._id)))
  setSelectedTask([])
}

return {
  tasks,
  selectedTask,
  setSelectedTask,
  onEditSubmit,
  handleDelete,
```

```
    handleComplete,  
  }  
}
```

Por que um hook customizado? Hooks são o padrão React para encapsular lógica com estado. O prefixo `use` é obrigatório para que o React reconheça como hook e aplique as regras corretamente. Tudo que era estado + efeito + handlers no `TaskList` agora vive aqui.

Extração 2 — Componente `EditTaskModal`

Arquivo: `src/components/taskBook/edit-task-modal.jsx`

O `useForm`, o schema zod e o JSX do `<dialog>` saem do `TaskList` e ficam dentro de quem os usa de verdade: o próprio modal.

```
import { useEffect } from "react"  
import { useForm } from "react-hook-form"  
import { zodResolver } from "@hookform/resolvers/zod"  
import { z } from "zod"  
import { PencilLine, X } from "lucide-react"  
  
const schema = z.object({  
  title: z.string().min(1, "Informe o novo título da tarefa"),  
  description: z.string().min(1, "Digite a nova descrição da tarefa"),  
})  
  
const EditTaskModal = ({ dialogRef, editingTask, onEditSubmit }) => {  
  const {  
    register,  
    handleSubmit,  
    formState: { errors },  
    reset,  
  } = useForm({ resolver: zodResolver(schema) })  
  
  useEffect(() => {  
    if (editingTask) {  
      reset({ title: editingTask.title, description: editingTask.description })  
    }  
  }, [editingTask, reset])  
  
  const onSubmit = async (data) => {  
    await onEditSubmit(editingTask, data)  
    dialogRef.current.close()  
  }  
  
  return (  
    <dialog ref={dialogRef} className="modal">  
      <div className="modal-box">  
        <div className="flex justify-between">  
          <div className="flex gap-2 mt-1">
```

```

        <PencilLine />
        <h3 className="font-bold text-lg mb-4">Editar Tarefa</h3>
      </div>
      <button
        type="button"
        className="btn btn-sm btn-circle rounded-full"
        onClick={() => dialogRef.current.close()}
      >
        <X />
      </button>
    </div>

    <form
      onSubmit={handleSubmit(onSubmit)}
      className="flex flex-col space-y-4"
      noValidate
    >
      <div>
        <h2>Novo título</h2>
        <input
          {...register("title")}
          type="text"
          placeholder="Título"
          className="input input-bordered w-full"
        />
        {errors.title && (
          <p className="text-error text-sm mt-1">{errors.title.message}</p>
        )}
      </div>

      <div>
        <h2>Nova descrição</h2>
        <input
          {...register("description")}
          type="text"
          placeholder="Descrição"
          className="input input-bordered w-full"
        />
        {errors.description && (
          <p className="text-error text-sm mt-1">{errors.description.message}
        </p>
        )}
      </div>

      <button
        type="submit"
        className="btn btn-success text-white rounded-full mt-2"
      >
        Salvar
      </button>
    </form>
  </div>

  <form method="dialog" className="modal-backdrop">

```

```
        <button>fechar</button>
      </form>
    </dialog>
  )
}

export default EditTaskModal
```

Por que o `useEffect` observa `editingTask`? No código original, o `reset({ title, description })` era chamado dentro do `handleEdit` antes de abrir o modal. Agora que o `useForm` vive dentro do `EditTaskModal`, o componente precisa saber quando a tarefa selecionada muda para resetar o form. O `useEffect` faz exatamente isso: toda vez que `editingTask` recebe um valor novo (usuário clica em editar outra tarefa), o form é atualizado com os dados corretos.

Props recebidas:

- `dialogRef` — referência ao `<dialog>`, controlada pelo `TaskList`
- `editingTask` — objeto da tarefa sendo editada (título + descrição para pré-preencher)
- `onEditSubmit` — função do hook `useTasks` que faz o PATCH na API

Resultado — `TaskList` simplificado

Arquivo: `src/components/taskBook/task-list.jsx`

```
import { useRef, useState } from "react"
import { PencilLine, Trash2 } from "lucide-react"
import { useTasks } from "../../hooks/use-tasks"
import EditTaskModal from "../edit-task-modal"

const TaskList = () => {
  const [editingTask, setEditingTask] = useState(null)
  const dialogRef = useRef(null)

  const {
    tasks,
    selectedTask,
    setSelectedTask,
    onEditSubmit,
    handleDelete,
    handleComplete,
  } = useTasks()

  const handleEdit = (task) => {
    setEditingTask(task)
    dialogRef.current.showModal()
  }

  return (
    <div className="min-w-[500px]">
      <ul className="list rounded-box space-y-4">
```

x1"

```

<li className="text-xs opacity-60 tracking-wide ml-14 p-4 pb-2">
  Lista de Tarefas
</li>

{tasks.map((task, index) => (
  <li key={task._id} className="flex items-center gap-4">
    <button
      className="btn btn-sm text-yellow-500 border-3 border-solid rounded-
xl"
      onClick={() => handleEdit(task)}
    >
      <PencilLine />
    </button>
    <div className="list-row flex-1 bg-emerald-600">
      <div className="text-4xl font-thin opacity-30 tabular-nums">
        {String(index + 1).padStart(2, "0")}
      </div>
      <div className="list-col-grow">
        <h1 className="font-bold uppercase">{task.title}</h1>
        <h3 className="text-xs font-semibold opacity-60">
          {task.description}
        </h3>
      </div>
      <div className="mt-2">
        <input
          className="checkbox checkbox-neutral"
          type="checkbox"
          checked={selectedTask.includes(task._id)}
          onChange={() =>
            setSelectedTask((current) =>
              current.includes(task._id)
                ? current.filter((id) => id !== task._id)
                : [...current, task._id]
            )
          }
        />
      </div>
    </div>
    <button
      className="btn btn-sm text-red-500 border-3 border-solid rounded-xl"
      onClick={() => handleDelete(task._id)}
    >
      <Trash2 />
    </button>
  </li>
)
)}}
</ul>

<div className="flex mt-4 justify-center">
  <button
    disabled={selectedTask.length === 0}
    className="btn btn-success rounded-md w-[150px] ml-19"
    onClick={handleComplete}
  >

```

```
        Concluir
      </button>
    </div>

    <EditTaskModal
      dialogRef={dialogRef}
      editingTask={editingTask}
      onEditSubmit={onEditSubmit}
    />
  </div>
)
}

export default TaskList
```

Comparativo antes e depois

Arquivo	Antes	Depois
task-list.jsx	~220 linhas	~70 linhas
edit-task-modal.jsx	não existia	~70 linhas
use-tasks.js	não existia	~60 linhas

O total de linhas aumenta levemente, mas cada arquivo tem uma única responsabilidade e pode ser alterado, testado e entendido de forma isolada.